

DSplit连通区域面积周长分析系统

基于C++与MFC的图像处理

Storm Lowe • 2026年6月

项目背景与目标:从理论到实践的图像处理探索

项目背景

基于C语言与MFC框架开发的图像处理应用程序
综合考察数据类型、运算符、控制语句、函数与模块化编程
涵盖数组与指针、字符串处理、内存管理、结构体与文件
操作
涉及图像分割、连通性分析和区域特征提取等核心算法



目标一

阈值分割:实现基于灰度值的图像二值化处理,将8位灰度图像转换为二值图像。有效区分目标物体与背景

目标二

连通区域检测:
在二值图像基础上识别并标记所有连通区域,准确统计图像中独立目标的个数

目标三

面积计算:
对每个连通区域计算所占像素数量,作为区域面积的基本度量

目标四

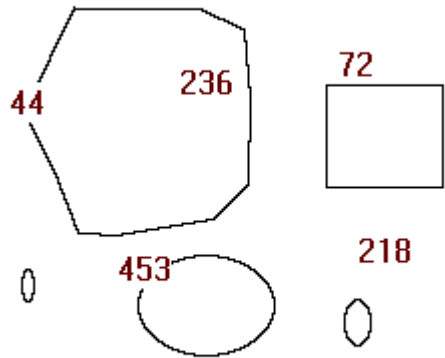
小区域消除:
基于面积阈值过滤噪声或无关小区域,保留有效目标区域

掌握图像处理核心算法,实现从原始图像到特征提取的完整流程

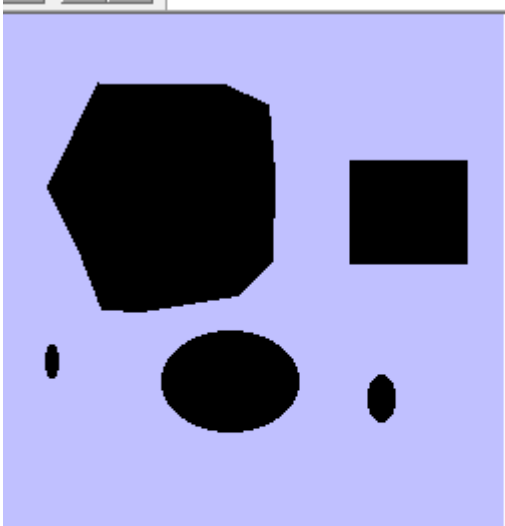
案例一：



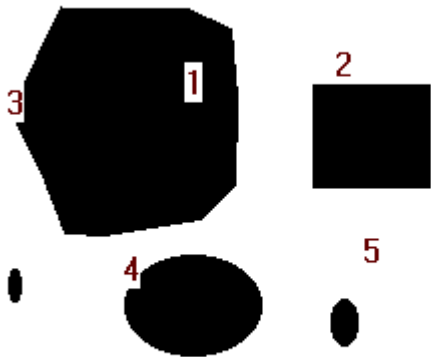
面积



周长

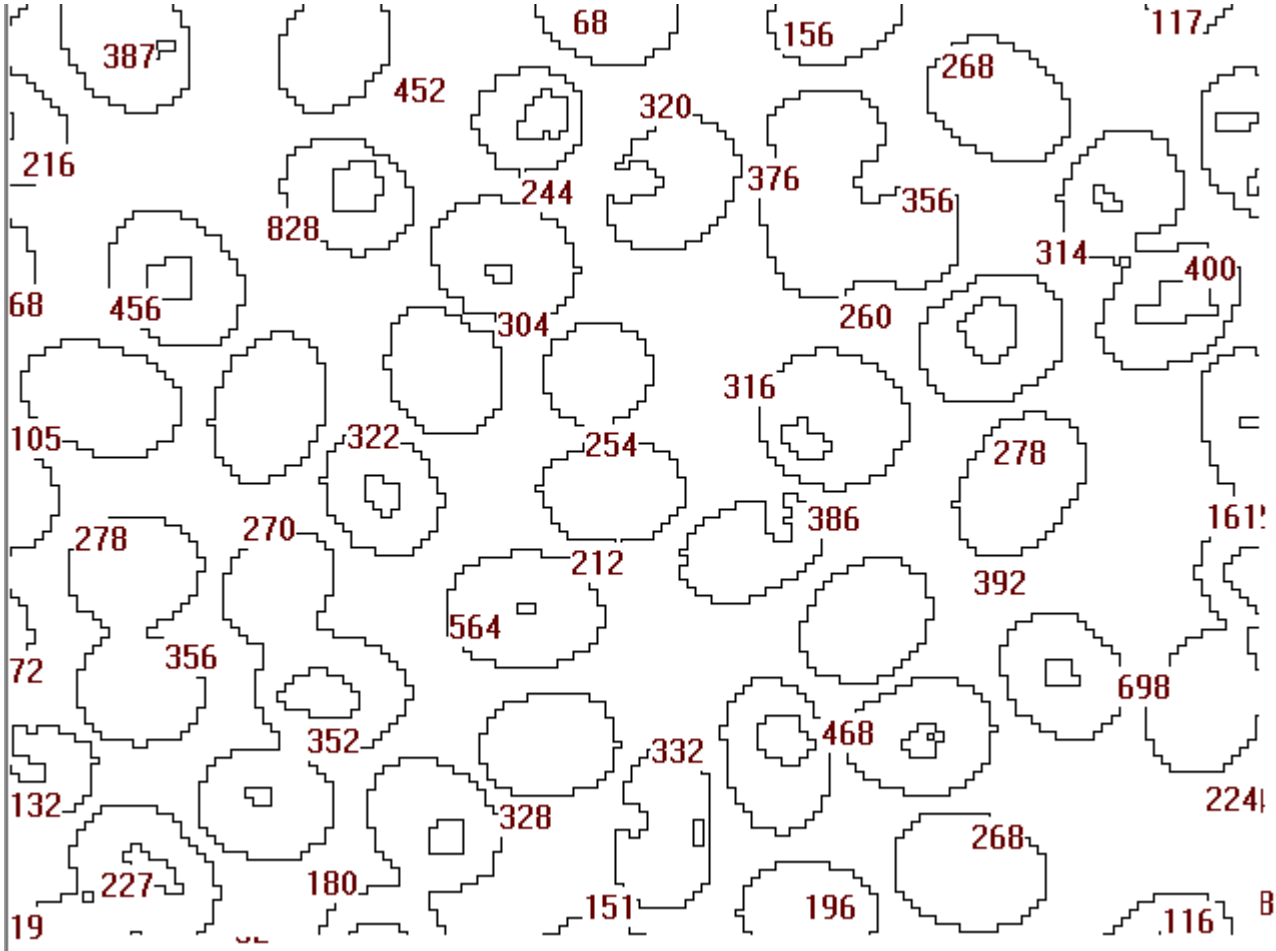


小区域消除

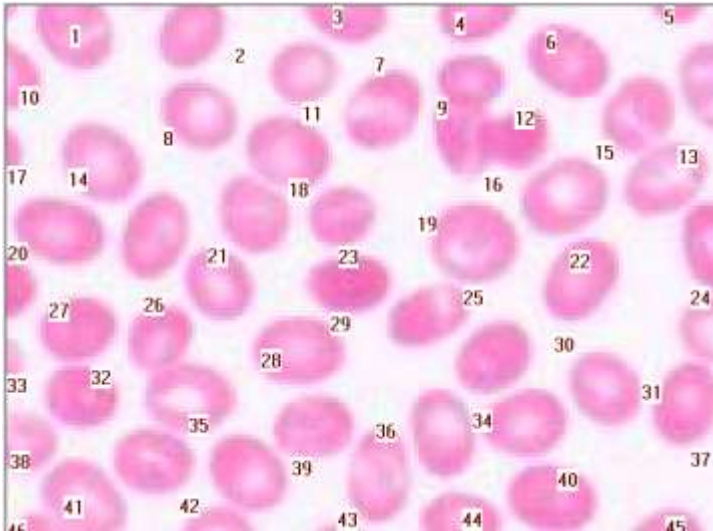


连通区标记

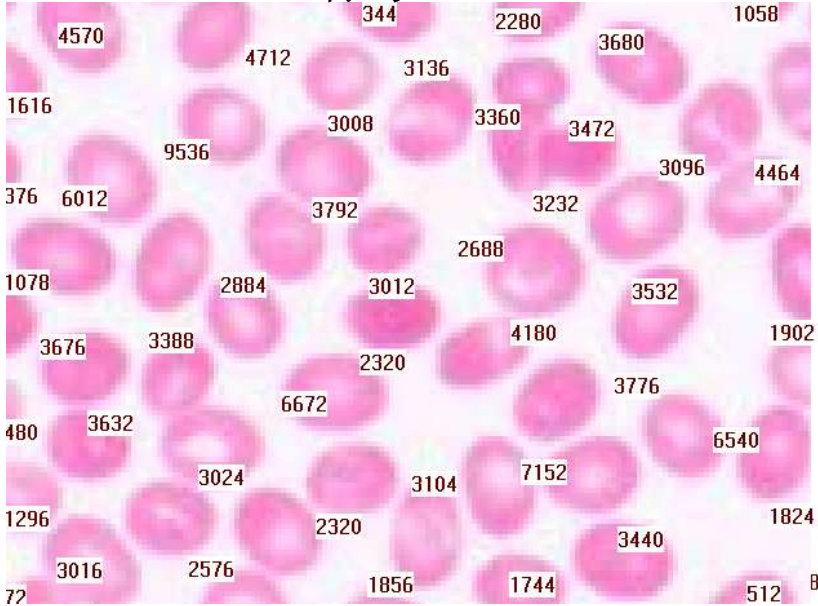
案例二：



周长



标号

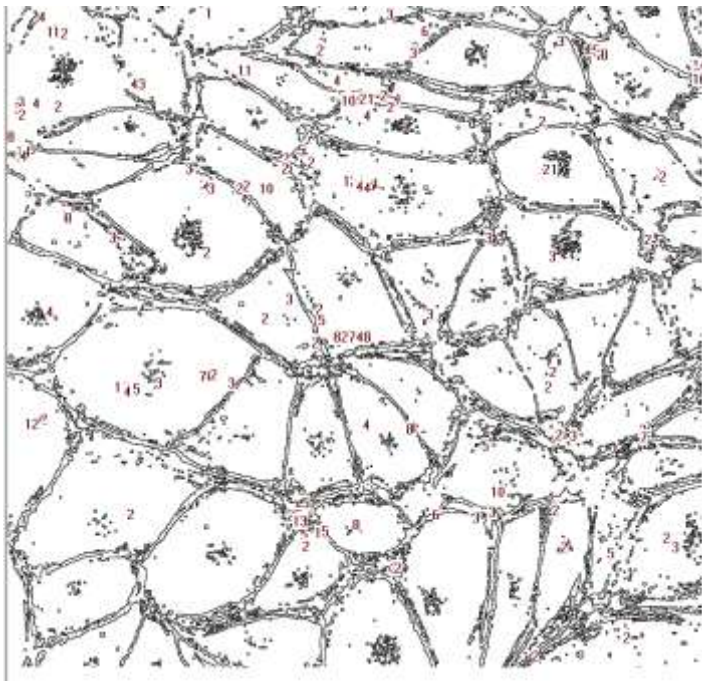
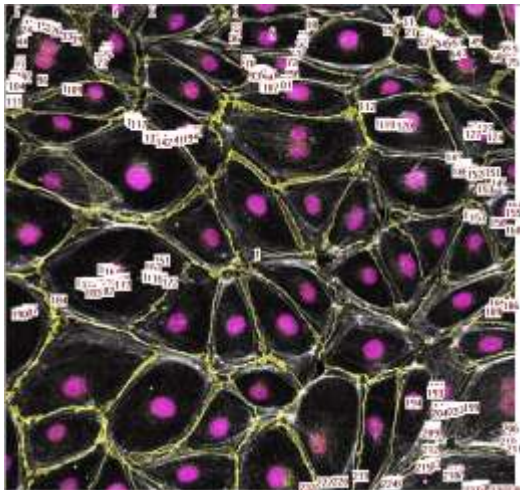


面积

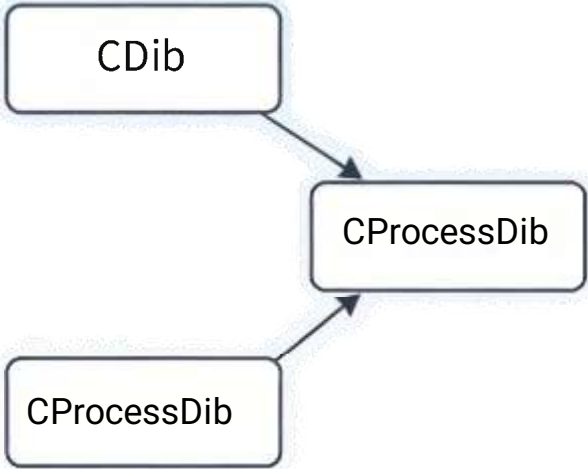
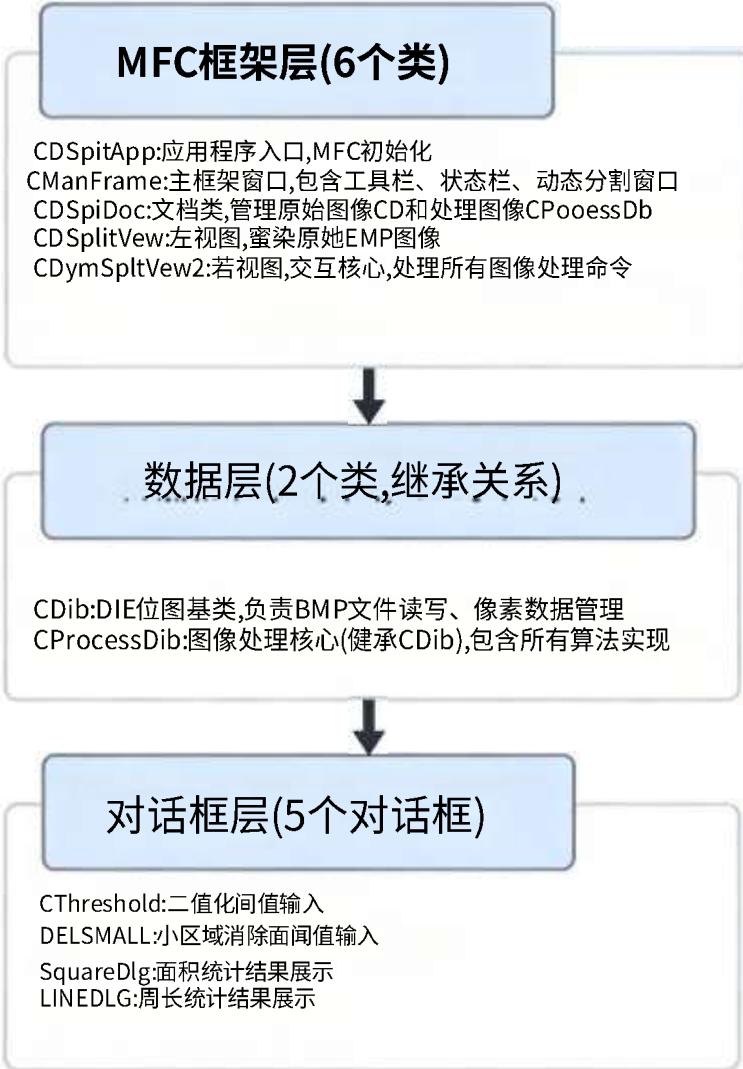
案例三：



案例四:



系统架构:三层设计,职责分明



核心算法一:二值化处理--将图像转化为黑白世界



算法原理

固定阈值二值化方法

灰度图:逐像素与阈值比较, $\geq$ 阈值翻白色(255),
 $<$ 阈值翻黑色(0)

24位彩色图:先用加权公式 $\text{gray} = (30R + 59G + 11B) / 100$ 转为灰度,再二值化

加权系数基于人眼对绿光敏感、对蓝光不敏感的原理

处理流程

- 1 步骤1「读取像素数据」:获取p_data数据区指针、图像宽高信息
- 2 步骤2「判断图像类型」:
根据biBitCount判断是灰度图还是24位彩色图
- 3 步骤3「灰度图处理」:
直接遍历像素,逐个与阈值比较进行二值化
- 4 步骤3「彩色图处理」:
先计算加权灰度值,再进行二值化,三通道统一
- 5 步骤4「输出结果」:原图像素数据被修改为二值图像

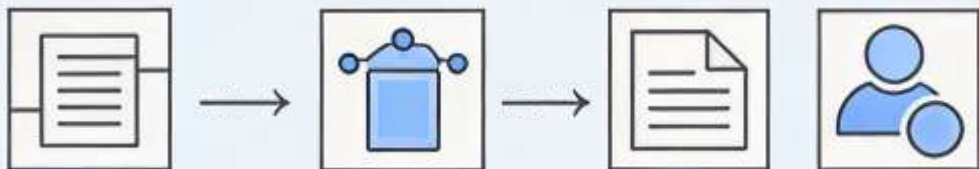
核心算法二:连通区域标记--两种算法的实现与对比

算法一:DFS栈式标记

算法类型:深度优先搜索(DFS)+栈结构

连通方式:4连通(上下左右)

实现方式:逐像素扫描,发现未标记前景点时使用vector作为栈进行DFS



数据结构:栈存储待访问像素坐标, p_temp存储标签图

内存特点:栈可能较大,递归深度大时存在栈溢出风险

应用场景: ClearSMALL小区域消除、Borderline周长统计

算法二:Two-Pass并查集

算法类型:两遍扫描+并查集(Union-Find)

连通方式:8连通(包含对角线方向)

实现方式:第一遍分配标签并合并等价类,第二遍扁平化生成连续编号

实现方式:并查集parent数组、labels
mapLabel映射数组

数据结构:并查集parent数组、labels标签数组、mapLabel映射数组

内存特点: $O(n)$ 并查集数组,无栈溢出风险,效率更高

应用场景:ConnectedRegions面积统计、编统计、编号标记

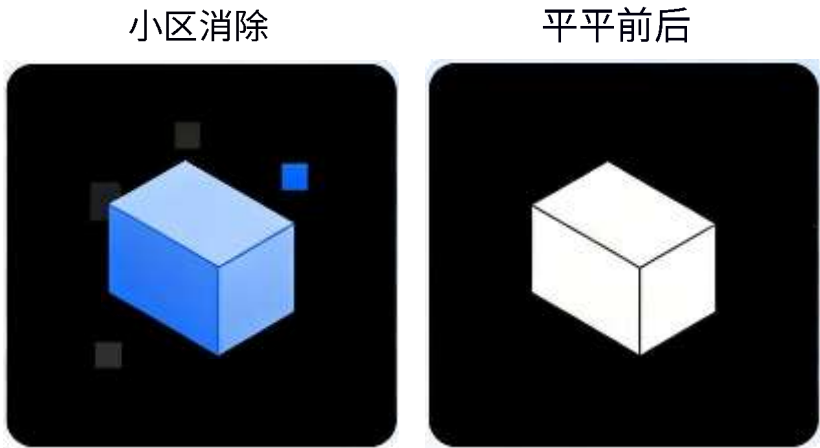
两种算法各有优劣,根据场景选择:DFS适合简单场景,Two-Pass效率更高

核心算法三：

面积计算与小区域消除--精准过滤噪声干扰

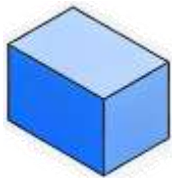
面积计算

- 在连通区域标记过程中同步统计
- 每个区域的面积=该标签对应的像素数量
- 使用flag[255]数组存储各区域面积
- 支持最多250个连通区域
- 计算公式:flaglabel+每遇到一个属于该标签的像素



小区域消除

标记连通区域:调用LabelNum()进行DFS标记,生成标签图p_temp



- ① 标历所有区域:
检查每个区域的面积flag[i]
- ② 遍历所有区域:
检查每个区域的面积flag[i]
- ③ 判断面积阈值:
若flag[u<m_value,则该区域为噪声小区域
- ④ 消除小区域:
遍历全图,将匹配区域编号的像素全部置为白色(255)
- ⑤ 输出结果:
保留大区域形态特征,去除噪声干扰

参数:m_value为面积阈值,用户可根据图像特点调整

核心算法四:边界提取与周长统计--精准刻画区域轮廓

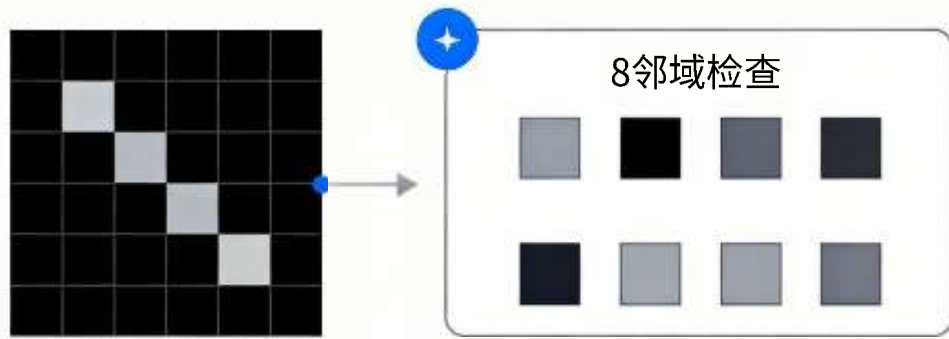
边界提取算法(8邻域法)

算法原理:遍历每个前景像素(黑色=0),检查其8邻域;

判断条件:
若当前像素为前景,且8邻域中存在非前景像素,则该像素为边界点;

实现细节:跳过图像边界(避免越界)
遍历y从1到height-2,x从1到width-2;
对每个前景像素,检查8个方向的邻居;若发现非前景邻居,标记为边界点(值为0);使用临时缓冲区避免边处理边读取导致的干扰;

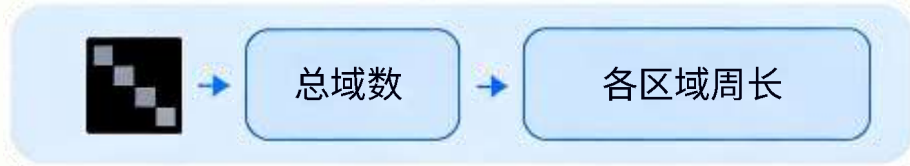
24位彩色图:三通道都为0视为前景黑色



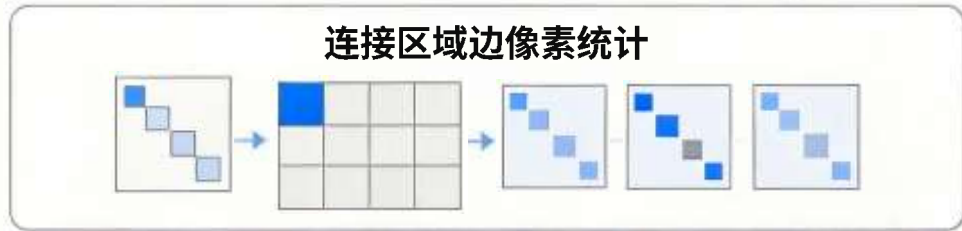
周长统计

基于标签图p_temp统计每个连通区域的边界像素数;
使用fn[255]数组存储各区域周长

使用fn[255]数组的各区域周长;
周长=该区域的边界像素数量;
总周长=所有区域边界像素之和



总周长=所有区域边界像素之和;
使用LINEDLG对话框展示结果:区域数、
总周长、各区域周长



数据结构:ppppl.pp_line存储各区域周长,供视图标注使用

数据处理流程:

从原始图像到特征提取的完整链路



技术亮点与难点攻克:关键数据结构的巧妙设计

p_data指针
原图像素数据指针
通过GetData()获取

支持灰度图和24位彩色图
索引计算:灰度图 $p_data[y]*DibWidth+x]$,
24位图 $p_data[y]*DibWidth+x*3]$

p_temp标签图
临时标签图,存储连通区域编号

255=背景/未标记,1~x_sign=区域编号
使用wide(像素宽)而非DibWidth索引

核心设计思想

flag面积数组



在标记过程中同步统计

双图像管理:原图CDib(只读)
+工作副本CProcessDi(可修改)
4字节对齐:DibWidthBytes()处理BMP行对齐要求
内存管理:SaveOrigin0备份、clearmem0恢复,避免数据污染

技术难点

像素索引计算的准确性:灰度图与24位图的索引方式不同
内存泄漏防范:及时释放临时缓冲区和标签图
溢出保护:xsign>250时提示并停止

pppp对象数组



存储连通区域的完整信息
包含pp_x.p_y坐标、pp_area面积、
pp_lin周长、pp_number编号
用于视图层的标注显示

项目成果与总结:完整实现图像处理核心功能



已实现功能

- ✓ 阈值分割:支持灰度图和24位彩色图的二值化处理
- ✓ 连通区域检测:实现DFS栈式和Two-Pass并查集两种算法
- ✓ 面积计算:准确统计每个连通区域的像素数量
- ✓ 小区域消除:基于面积阈值过滤噪声,保留有效目标
- ✓ 周长统计:8邻域边界提取,计算各区域轮廓长度
- ✓ 编号标记:为每个连通区域分配唯一编号并可视化展示
- ✓ 结果保存:支持将处理结果保存为BMP文件

技术收获

深入理解BMP图像格式:文件头、信息头、颜色表、像素数据结构

掌握图像处理核心算法:二值化、连通标记、边界提取

实践MFC框架开发:文档-视图架构、消息映射、对话框交互

项目文件结构

源代码12对(.cpp/.h)|资源文件4个

测试图片3张|项目配置文件

